



UNIVERSIDAD NACIONAL AUTÓNOMA DE
MÉXICO

FACULTAD DE INGENIERÍA
DIVISIÓN DE INGENIERÍA ELÉCTRICA
INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA
e INTERACCIÓN HUMANO COMPUTADORA



EJERCICIOS DE CLASE N° 02

NOMBRE COMPLETO: Gómez Aceves Abraham Agustín

N° de Cuenta: 320093290

GRUPO DE LABORATORIO: 03

GRUPO DE TEORÍA: 05

SEMESTRE 2027-1

FECHA DE ENTREGA LÍMITE: 01 – septiembre - 2026

CALIFICACIÓN: _____

EJERCICIOS DE SESIÓN:

1. Actividades realizadas. Una descripción de los ejercicios y capturas de pantalla de bloques de código generados y de ejecución del programa
2. Problemas presentados. Listar si surgieron problemas a la hora de ejecutar el código
3. Conclusión:
 - a. Los ejercicios de la clase: Complejidad, explicación
 - b. Comentarios generales: Faltó explicar a detalle, ir más lento en alguna explicación, otros comentarios y sugerencias.

1.- Actividades realizadas y pruebas

1.- generar las figuras copiando los vértices de triangulorojo y cuadradoverde:

-triángulo azul

-triangulo verde (0,0.5,0)

-cuadrado rojo

-cuadrado verde

-cuadrado café (0.478, 0.255, 0.067)

2.- Usando la proyección ortogonal generar el siguiente dibujo a partir de instancias de las figuras anteriormente creadas , recordar que todos se dibujan en el origen y por transformaciones geométricas se desplazan

Secciones modificadas del codigo
Debido a la cantidad de figuras requeridas para el ejercicio, fue necesario reemplazar casi por completo el código anterior en favor de añadir nuevo para que saliera correctamente
<pre>//Práctica 2: Índices, mesh, proyecciones, transformaciones geométricas #include <stdio.h> #include <string.h> #include #include #include <glew.h> #include <glfw3.h> //glm #include<glm.hpp> #include<gtc\matrix_transform.hpp> #include<gtc\type_ptr.hpp> //clases para dar orden y limpieza al código #include"Mesh.h" #include"Shader.h" #include"Window.h" //Dimensiones</pre>

```

de la ventana const float toRadians = 3.14159265f / 180.0; //grados a
radianes Window mainWindow; std::vector<Mesh*>
meshList; std::vector<MeshColor*> meshColorList; std::vector<shaderList>; //Vertex
Shader static const char* vShader = "shaders/shader.vert"; static const char*
fShader = "shaders/shader.frag"; static const char* vShaderColor =
"shaders/shadercolor.vert"; static const char* fShaderColor =
"shaders/shadercolor.frag";

float angulo = 0.0f;

//color café en RGB : 0.478, 0.255, 0.067

//Pirámide triangular regular void CreaPiramide() { unsigned int indices[] =
{ 0,1,2, 1,3,2, 3,0,2, 1,0,3 }; GLfloat vertices[] = { -0.5f,
-0.5f,0.0f, 0.5f,-0.5f,0.0f, 0.0f,0.5f, -0.25f, 0.0f,-0.5f,-0.5f, }; Mesh* obj1 = new
Mesh(); obj1->CreateMesh(vertices, indices, 12, 12); meshList.push_back(obj1); }

//Vértices de un cubo void CrearCubo() { unsigned int cubo_indices[] = { 0, 1, 2, 2,
3, 0, 1, 5, 6, 6, 2, 1, 7, 6, 5, 5, 4, 7, 4, 0, 3, 3, 7, 4, 4, 5, 1, 1, 0, 4, 3, 2, 6, 6, 7,
3 }; GLfloat cubo_vertices[] = { -0.5f, -0.5f, 0.5f, 0.5f, -0.5f, 0.5f, 0.5f, 0.5f,
0.5f, -0.5f, 0.5f, 0.5f, -0.5f, -0.5f, -0.5f, 0.5f, -0.5f, -0.5f, 0.5f, 0.5f, -0.5f, -0.5f,
0.5f, -0.5f }; Mesh* cubo = new Mesh(); cubo->CreateMesh(cubo_vertices,
cubo_indices, 24, 36); meshList.push_back(cubo); }

// Índices en meshColorList al terminar esta función: // 0 letras (sin usar) 1
triangulorojo 2 cuadradoverde // 3 trianguloazul 4 trianguloverde 5
trianguloamarillo // 6 trianguloPurpura 7 cuadradoverde 8 cuadradoverde // 9
cuadradoverde (piso) // OJO: triangulo* tiene medio-ancho = 1 (vertices en
-1..1) // cuadrado* tiene medio-ancho = 0.5 (vertices en -0.5..0.5) void
CrearLetrasyFiguras() { GLfloat vertices_letras[] = { -1.0f, -1.0f, 0.5f,
0.0f, 0.0f, 1.0f, 1.0f, -1.0f, 0.5f, 0.0f,
0.0f, 1.0f, 1.0f, 1.0f, 0.5f, 0.0f, 0.0f, 1.0f, };
MeshColor* letras = new MeshColor(); letras->CreateMeshColor(vertices_letras,
18); meshColorList.push_back(letras); // 0

GLfloat vertices_triangulorojo[] = {
-1.0f, -1.0f, 0.5f, 1.0f, 0.0f, 0.0f,
1.0f, -1.0f, 0.5f, 1.0f, 0.0f, 0.0f,
0.0f, 1.0f, 0.5f, 1.0f, 0.0f, 0.0f,
};
MeshColor* triangulorojo = new MeshColor();
triangulorojo->CreateMeshColor(vertices_triangulorojo, 18);
meshColorList.push_back(triangulorojo); // 1

```

```

GLfloat vertices_cuadradoverde[] = {
    -0.5f,    -0.5f,    0.5f,    0.0f,  1.0f,  0.0f,
    0.5f,    -0.5f,    0.5f,    0.0f,  1.0f,  0.0f,
    0.5f,    0.5f,    0.5f,    0.0f,  1.0f,  0.0f,
    -0.5f,    -0.5f,    0.5f,    0.0f,  1.0f,  0.0f,
    0.5f,    0.5f,    0.5f,    0.0f,  1.0f,  0.0f,
    -0.5f,    0.5f,    0.5f,    0.0f,  1.0f,  0.0f,
};
MeshColor* cuadradoverde = new MeshColor();
cuadradoverde->CreateMeshColor(vertices_cuadradoverde, 36);
meshColorList.push_back(cuadradoverde); // 2

GLfloat vertices_trianguoloazul[] = {
    -1.0f, -1.0f, 0.5f,  0.0f, 0.0f, 1.0f,
    1.0f, -1.0f, 0.5f,  0.0f, 0.0f, 1.0f,
    0.0f, 1.0f, 0.5f,  0.0f, 0.0f, 1.0f,
};
MeshColor* trianguoloazul = new MeshColor();
trianguoloazul->CreateMeshColor(vertices_trianguoloazul, 18);
meshColorList.push_back(trianguoloazul); // 3

GLfloat vertices_trianguoverde[] = {
    -1.0f, -1.0f, 0.5f,  0.0f, 0.5f, 0.0f,
    1.0f, -1.0f, 0.5f,  0.0f, 0.5f, 0.0f,
    0.0f, 1.0f, 0.5f,  0.0f, 0.5f, 0.0f,
};
MeshColor* trianguoverde = new MeshColor();
trianguoverde->CreateMeshColor(vertices_trianguoverde, 18);
meshColorList.push_back(trianguoverde); // 4

GLfloat vertices_trianguoamarillo[] = {
    -1.0f, -1.0f, 0.5f,  1.0f, 1.0f, 0.0f,
    1.0f, -1.0f, 0.5f,  1.0f, 1.0f, 0.0f,
    0.0f, 1.0f, 0.5f,  1.0f, 1.0f, 0.0f,
};
MeshColor* trianguoamarillo = new MeshColor();
trianguoamarillo->CreateMeshColor(vertices_trianguoamarillo, 18);
meshColorList.push_back(trianguoamarillo); // 5

GLfloat vertices_trianguoPurpura[] = {
    -1.0f, -1.0f, 0.5f,  0.5f, 0.0f, 1.0f,
    1.0f, -1.0f, 0.5f,  0.5f, 0.0f, 1.0f,
    0.0f, 1.0f, 0.5f,  0.5f, 0.0f, 1.0f,
};

```

```

};
MeshColor* trianguloPurpura = new MeshColor();
trianguloPurpura->CreateMeshColor(vertices_trianguloPurpura, 18);
meshColorList.push_back(trianguloPurpura); // 6

GLfloat vertices_cuadradorojo[] = {
    -0.5f, -0.5f, 0.5f,  1.0f, 0.0f, 0.0f,
    0.5f, -0.5f, 0.5f,  1.0f, 0.0f, 0.0f,
    0.5f, 0.5f, 0.5f,  1.0f, 0.0f, 0.0f,
    -0.5f, -0.5f, 0.5f,  1.0f, 0.0f, 0.0f,
    0.5f, 0.5f, 0.5f,  1.0f, 0.0f, 0.0f,
    -0.5f, 0.5f, 0.5f,  1.0f, 0.0f, 0.0f,
};
MeshColor* cuadradorojo = new MeshColor();
cuadradorojo->CreateMeshColor(vertices_cuadradorojo, 36);
meshColorList.push_back(cuadradorojo); // 7

GLfloat vertices_cuadrado cafe[] = {
    -0.5f, -0.5f, 0.5f,  0.478f, 0.255f, 0.067f,
    0.5f, -0.5f, 0.5f,  0.478f, 0.255f, 0.067f,
    0.5f, 0.5f, 0.5f,  0.478f, 0.255f, 0.067f,
    -0.5f, -0.5f, 0.5f,  0.478f, 0.255f, 0.067f,
    0.5f, 0.5f, 0.5f,  0.478f, 0.255f, 0.067f,
    -0.5f, 0.5f, 0.5f,  0.478f, 0.255f, 0.067f,
};
MeshColor* cuadrado cafe = new MeshColor();
cuadrado cafe->CreateMeshColor(vertices_cuadrado cafe, 36);
meshColorList.push_back(cuadrado cafe); // 8

GLfloat vertices_cuadrado negro[] = {
    -0.5f, -0.5f, 0.5f,  0.0f, 0.0f, 0.0f,
    0.5f, -0.5f, 0.5f,  0.0f, 0.0f, 0.0f,
    0.5f, 0.5f, 0.5f,  0.0f, 0.0f, 0.0f,
    -0.5f, -0.5f, 0.5f,  0.0f, 0.0f, 0.0f,
    0.5f, 0.5f, 0.5f,  0.0f, 0.0f, 0.0f,
    -0.5f, 0.5f, 0.5f,  0.0f, 0.0f, 0.0f,
};
MeshColor* cuadrado negro = new MeshColor();
cuadrado negro->CreateMeshColor(vertices_cuadrado negro, 36);
meshColorList.push_back(cuadrado negro); // 9
}

void CreateShaders() { Shader* shader1 = new Shader(); shader1-

```

```

>CreateFromFiles(vShader, fShader); shaderList.push_back(*shader1);

Shader* shader2 = new Shader();
shader2->CreateFromFiles(vShaderColor, fShaderColor);
shaderList.push_back(*shader2);
}

void DibujarPieza(MeshColor* mesh, GLuint uniformModel, GLuint
uniformProjection, const glm::mat4& projection, glm::vec3 figuraOffset, float
anguloGrados, glm::vec3 traslacionLocal, glm::vec3 escala) { glm::mat4 model =
glm::mat4(1.0f); model = glm::translate(model, figuraOffset); if (anguloGrados !=
0.0f) model = glm::rotate(model, glm::radians(anguloGrados), glm::vec3(0.0f, 0.0f,
1.0f)); model = glm::translate(model, traslacionLocal); model = glm::scale(model,
escala);

glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
mesh->RenderMeshColor();
}

int main() { mainWindow = Window(800, 600); mainWindow.Initialise();

//Escena 2D "por capas": desactivamos el depth test para que el orden de
//dibujo (pintor) decida qué queda encima, aunque todo esté al mismo Z.
glDisable(GL_DEPTH_TEST);

CreaPiramide();
CrearCubo();
CrearLetrasyFiguras();
CreateShaders();
GLuint uniformProjection = 0;
GLuint uniformModel = 0;

// Ortho con la misma relación de aspecto que la ventana (800/600 = 4/3)
glm::mat4 projection = glm::ortho(-4.0f, 4.0f, -3.0f, 3.0f, 0.1f, 100.0f);

glm::mat4 model(1.0);

while (!mainWindow.getShouldClose())
{
    glfwPollEvents();
    glClearColor(1.0f, 1.0f, 1.0f, 1.0f); // fondo blanco
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);

    shaderList[1].useShader();
}

```

```

uniformModel = shaderList[1].getModelLocation();
uniformProjection = shaderList[1].getProjectLocation();

glm::vec3 offset1(-2.5f, 0.0f, -4.0f); // figura 1 (barras)
glm::vec3 offset2(0.0f, 0.0f, -4.0f); // figura 2 (cuadrado + rombos)
glm::vec3 offset3(2.5f, 0.0f, -4.0f); // figura 3 (Sierpinski)
glm::vec3 offsetGlobal(0.0f, 0.0f, -4.0f); // para piezas que no pertenecen a una
sola figura (piso)

// ===== PISO NEGRO: toca ambos extremos de la
pantalla =====
// ortho va de -4 a 4 en X -> medio-ancho necesario = 4 -> escala.x = 4 / 0.5 =
8
DibujarPieza(meshColorList[9], uniformModel, uniformProjection, projection,
offsetGlobal, 0.0f, glm::vec3(0.0f, -1.36f, 0.0f), glm::vec3(8.0f, 0.30f, 1.0f));

// ===== FIGURA 1: barras + triángulos invertidos
=====
// Triángulos: base=0.45 de medio-ancho -> sus vértices exteriores quedan en
x=-0.45 y x=+0.45
DibujarPieza(meshColorList[5], uniformModel, uniformProjection, projection,
offset1, 0.0f, glm::vec3(0.0f, 0.30f, 0.0f), glm::vec3(0.45f, -0.30f, 1.0f)); //
amarillo
DibujarPieza(meshColorList[1], uniformModel, uniformProjection, projection,
offset1, 0.0f, glm::vec3(0.0f, -0.32f, 0.0f), glm::vec3(0.45f, -0.30f, 1.0f)); //
rojo
DibujarPieza(meshColorList[4], uniformModel, uniformProjection, projection,
offset1, 0.0f, glm::vec3(0.0f, -0.92f, 0.0f), glm::vec3(0.45f, -0.30f, 1.0f)); //
verde

// Barras café: su borde interior toca x=-0.45/+0.45 (vértice exterior de los
triángulos),
// suben más allá de la base del triángulo amarillo (y=0.92) y bajan hasta el
piso (y=-1.0)
DibujarPieza(meshColorList[8], uniformModel, uniformProjection, projection,
offset1, 0.0f, glm::vec3(-0.50f, -0.16f, 0.0f), glm::vec3(0.10f, 2.10f, 1.0f)); //
barra izq
DibujarPieza(meshColorList[8], uniformModel, uniformProjection, projection,
offset1, 0.0f, glm::vec3(0.50f, -0.16f, 0.0f), glm::vec3(0.10f, 2.10f, 1.0f)); //
barra der

// ===== FIGURA 2: cuadrado (4 triángulos) + rombo azul + rombo
café =====

```

```

    DibujarPieza(meshColorList[5], uniformModel, uniformProjection, projection,
        offset2, -135.0f, glm::vec3(0.0f, -1.21f, 0.0f), glm::vec3(1.0f, -0.5f, 1.0f)); //
    amarillo (arriba)
    DibujarPieza(meshColorList[1], uniformModel, uniformProjection, projection,
        offset2, 135.0f, glm::vec3(0.0f, -1.21f, 0.0f), glm::vec3(1.0f, -0.5f, 1.0f)); //
    rojo (derecha)
    DibujarPieza(meshColorList[4], uniformModel, uniformProjection, projection,
        offset2, 45.0f, glm::vec3(0.0f, -1.21f, 0.0f), glm::vec3(1.0f, -0.5f, 1.0f)); //
    verde (abajo)
    DibujarPieza(meshColorList[6], uniformModel, uniformProjection, projection,
        offset2, -45.0f, glm::vec3(0.0f, -1.21f, 0.0f), glm::vec3(1.0f, -0.5f, 1.0f)); //
    morado (izquierda)

    // Rombo azul (grande, centro): 2 mitades de triangulo azul; sus puntas tocan
    // los puntos medios de los 4 lados del cuadrado exterior
    DibujarPieza(meshColorList[3], uniformModel, uniformProjection, projection,
        offset2, 0.0f, glm::vec3(0.0f, 0.5f, 0.0f), glm::vec3(1.0f, 0.5f, 1.0f)); // mitad de
    arriba
    DibujarPieza(meshColorList[3], uniformModel, uniformProjection, projection,
        offset2, 0.0f, glm::vec3(0.0f, -0.5f, 0.0f), glm::vec3(1.0f, -0.5f, 1.0f)); // mitad
    de abajo

    // Rombo café (chico), centrado dentro del rombo azul (cuadrado rotado 45°)
    DibujarPieza(meshColorList[8], uniformModel, uniformProjection, projection,
        offset2, 45.0f, glm::vec3(0.0f, 0.0f, 0.0f), glm::vec3(0.45f, 0.45f, 1.0f));

    // ===== FIGURA 3: triángulo grande dividido en 4
    =====
    DibujarPieza(meshColorList[6], uniformModel, uniformProjection, projection,
        offset3, 0.0f, glm::vec3(0.0f, 0.28f, 0.0f), glm::vec3(0.5f, 0.5f, 1.0f)); // morado
    DibujarPieza(meshColorList[4], uniformModel, uniformProjection, projection,
        offset3, 0.0f, glm::vec3(-0.5f, -0.72f, 0.0f), glm::vec3(0.5f, 0.5f, 1.0f)); // verde
    DibujarPieza(meshColorList[1], uniformModel, uniformProjection, projection,
        offset3, 0.0f, glm::vec3(0.5f, -0.72f, 0.0f), glm::vec3(0.5f, 0.5f, 1.0f)); // rojo
    DibujarPieza(meshColorList[5], uniformModel, uniformProjection, projection,
        offset3, 0.0f, glm::vec3(0.0f, -0.72f, 0.0f), glm::vec3(0.5f, -0.5f, 1.0f)); //
    amarillo invertido

    glUseProgram(0);
    mainWindow.swapBuffers();
}
return 0;

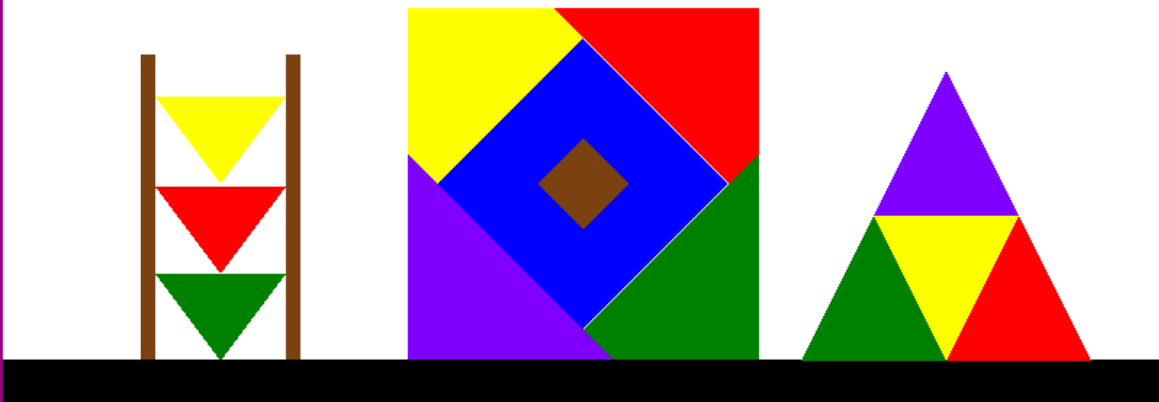
```



```
}
```

Prueba de funcionamiento del código

Practica 2: Proyecciones, transformaciones



2.- Problemas presentados

1. Dificultad para visualizar la escena sin referencias claras

Uno de los principales retos fue, sin duda, ajustar posiciones, tamaños y rotaciones sin tener una retroalimentación visual inmediata y clara. Los comentarios en el código reflejan que se hicieron cálculos manuales para determinar, por ejemplo, dónde terminaban los vértices de una figura. Esto indica que el proceso de ubicación espacial fue complejo y requirió bastante prueba y error.

2. Complejidad en el manejo de transformaciones

La combinación de traslaciones, rotaciones y escalas no es trivial, sobre todo cuando se trabaja con objetos que tienen dimensiones base distintas. Por ejemplo, los triángulos tienen un medio ancho de 1.0, mientras que los cuadrados lo tienen de 0.5. Esa diferencia obligó a realizar ajustes específicos para cada figura, lo cual volvió el código más difícil de mantener y de entender.

3. Uso de valores negativos en escalas

El empleo de escalas negativas para invertir figuras —como en el caso de los triángulos— es una técnica que no resulta intuitiva al principio. Muy probablemente se descubrió después de varios intentos, ya que su efecto no siempre es evidente cuando se está aprendiendo a trabajar con transformaciones geométricas.

4. Falta de una estructura más organizada

El código concentra demasiada lógica en la función principal. La creación de mallas, la configuración de shaders y el renderizado están mezclados, lo que dificulta tanto la lectura como la modificación. Conforme crecía la escena, mantener todo en un solo lugar seguramente se volvió cada vez más complicado.

3.- Conclusión

Este proyecto dejó en claro que hacer gráficos no es solo cuestión de escribir código, sino de entender cómo se comporta cada elemento en el espacio. A lo largo del desarrollo, fue necesario ajustar posiciones, tamaños y rotaciones a base de prueba y error, lo que enseñó la importancia de tener claridad sobre las coordenadas desde el inicio. También se aprendió que las transformaciones no siempre son intuitivas, y que detalles como el orden de las operaciones o el uso de escalas negativas pueden cambiar por completo el resultado visual. La falta de una estructura más ordenada dificultó los cambios sobre la marcha, y los valores escritos directamente complicaron recordar qué representaba cada número. Aun así, el proceso sirvió para comprender mejor el funcionamiento del pipeline gráfico.

y la necesidad de mantener el código organizado. En general, fue una experiencia valiosa que mostró tanto los avances logrados como los aspectos que aún pueden mejorarse.